

Trường Đại học Công nghệ Thông tin

Đại học Quốc gia Tp.Hồ Chí Minh

Toàn diện – Sáng tạo - Phụng sự



NỘP BÀI TẬP THỰC HÀNH BUỔI 3

Môn học: Kỹ thuật xây dựng hệ thống Agentic AI (SE373)

Bài tập: Dựng agent đặt vé bằng LangChain

Họ và tên sinh viên: Trần Minh Thuận

Mã số sinh viên: 24521746

1) THIẾT KẾ CÁC LỚP HARNESS BẢO VỆ

- Trong kiến trúc Agentic Engineering, hệ thống tuân thủ công thức:

$$\text{Agent} = \text{Goal} + \text{Tools} + \text{Loop} + \text{Termination}$$

- Harness đóng vai trò là khung bao bọc bằng code cố định để kiểm soát hành vi, chi phí và sự an toàn của mô hình. **Hệ thống đã cài đặt đầy đủ 4 lớp Harness tiêu chuẩn:**

❖ Lớp 1: Ràng buộc là dữ liệu (constraints.py)

- **Mục đích:** Thay vì để các yêu cầu đặt vé nằm rải rác dưới dạng câu lệnh prompt tự nhiên, toàn bộ tiêu chí người dùng được cấu trúc hóa thành lớp dữ liệu BookingConstraint (origin, dest, date, max_price, depart_before).
- **Tác dụng:** Giữ nguyên mục tiêu tại một vị trí cố định làm hệ quy chiếu cho toàn bộ quá trình kiểm định của Harness.

❖ Lớp 2: Kiểm quyền trước khi gọi tool (permission.py)

- **Vị trí chạy:** Chạy tại kiểm tra 0.
- **Logic can thiệp:**
 - Chặn hành động pay và yêu cầu con người phê duyệt.
 - Chặn hành động giữ chỗ nếu phát hiện vé không hoàn tiền (refundable == False) hoặc giá vé vượt trần ngân sách cho phép.
- **Tác dụng:** Ngăn chặn rủi ro tổn thất tài chính ngoài ý muốn khi Agent tự ý ra quyết định.

❖ Lớp 3: Tiêu chí hoàn thành kiểm bằng code (completion_sensor.py)

- **Bản chất:** Áp dụng Sensor computational thay vì dựa vào Sensor inferential .
- **Logic kiểm tra:** Logic kiểm chứng chéo trực tiếp trên trạng thái dữ liệu của hệ thống:

`status == "confirmed" ^ paid == True ^ price ≤ max_price ^ date_ok ^ time_ok`

- **Tác dụng:** Ngăn chặn lỗi ảo giác khi mô hình tự nhận là đã xong dù cơ sở dữ liệu chưa ghi nhận đặt chỗ thành công.

❖ Lớp 4: Cơ chế bàn giao cho con người (handoff.py) & Phát hiện lặp (loop_detector.py)

- **LoopDetector:** Lưu trữ dấu vân tay vào một hàng đợi trượt. Nếu cùng một hành vi lặp lại quá số lần quy định hoặc đại lượng tiến triển đứng yên, hệ thống sẽ ngắt luồng ngay lập tức để tiết kiệm token.
- **HandoffReport:** Khi bị ngắt do chạm quyền hoặc lỗi lặp, hệ thống xuất báo cáo tóm tắt cấu trúc: Trạng thái hiện tại, Các bước đã thử, Hành động gây lỗi, và Câu hỏi ngắn gọn giúp con người đưa ra quyết định xử lý chỉ trong 30 giây.

```

Run main x
D:\SE373\flight_booking_agent\.venv\Scripts\python.exe D:\SE373\flight_booking_agent\main.py
>>> KHỎI CHẠY KIỂM THỬ AGENT ĐẶT VÉ MÁY BAY <<<

Kết quả thực thi Agent: WAIT_APPROVAL

===== BÀN GIAO CHO CON NGƯỜI (HANDOFF) =====
[Trạng thái]      : CẦN PHÉ DUYỆT
[Giai đoạn dừng]  : Vòng 3 tại tool 'book_seat'
[Số bước đã thử]  : 2 bước
[Thất bại/Cảnh báo] : Không có
[Câu hỏi cụ thể]  : Vé không hoàn tiền (non-refundable). Cần phê duyệt trước khi đặt chỗ. Bạn có đồng ý thực thi không?
=====

Process finished with exit code 0

```

Hình 1: Minh chứng hoạt động của lớp kiểm quyền (PermissionGuard) và cơ chế Bàn giao (HandoffReport) khi chạy main.py

2) CÀI ĐẶT 3 MẪU THIẾT KẾ AGENT

❖ Mẫu 1: ReAct Agent (react_agent.py)

- **Nguyên lý:** Hoạt động tuần tự theo chu trình **Thought** → **Action** → **Observation**.
- **Cơ chế:** Dữ liệu từ môi trường sau mỗi bước đi vào giữa chuỗi suy luận, giúp Agent tự điều chỉnh hướng đi sau mỗi quan sát thực tế.
- **Rủi ro kiểm soát:** Được bọc bởi LoopDetector để ngăn tình trạng lặp vô hạn và trôi mục tiêu.

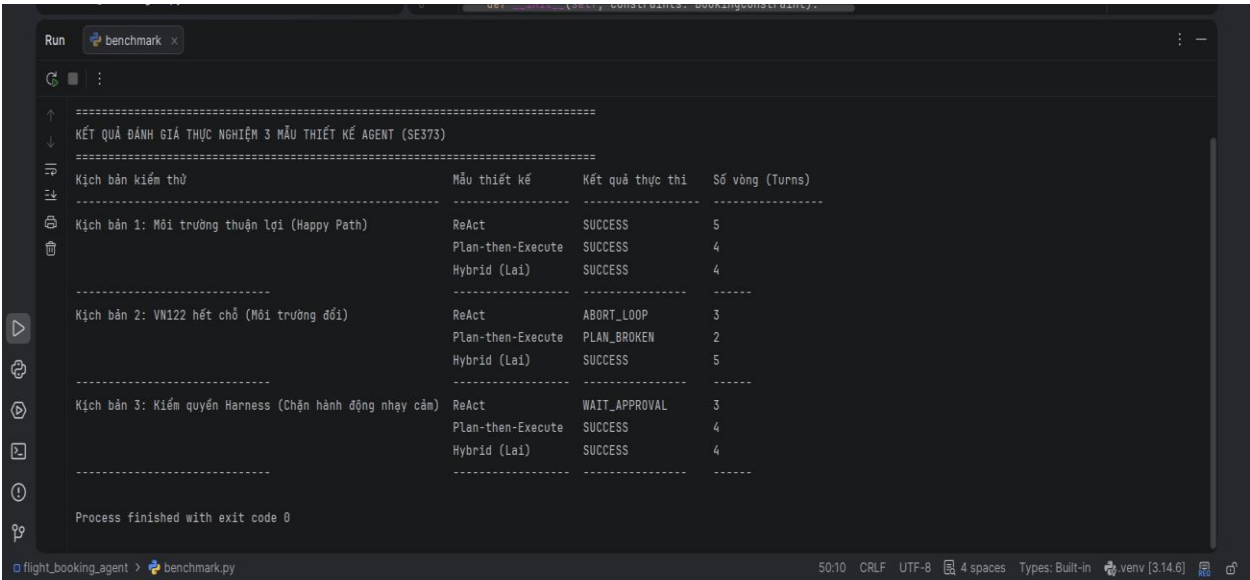
❖ Mẫu 2: Plan-then-Execute Agent (plan_execute_agent.py)

- **Nguyên lý:** Sinh trọn vẹn danh sách các bước cần làm ngay từ đầu thông qua hàm generate_static_plan(), sau đó thực thi tuần tự từng bước.
- **Ưu điểm:** Kế hoạch nhìn thấy được toàn bộ trước khi chạy, cho phép con người duyệt trước và ước tính được chi phí token.
- **Nhược điểm:** Thiếu tính thích nghi vì nếu một bước gặp trục trặc như hết vé thì toàn bộ kế hoạch phía sau sẽ bị gãy.

❖ **Mẫu 3: Mẫu Lai - Hybrid (hybrid_agent.py)**

- **Nguyên lý:** Kết hợp giữa lập kế hoạch khung và khả năng phản xạ ReAct.
- **Cơ chế Re-planning:** Hệ thống đặt ra các checkpoint. Khi bước kiểm tra ghế phát hiện chuyến bay đã hết chỗ hoặc giá vé không thỏa mãn, Agent không dừng lại mà lập tức kích hoạt nhánh Re-planning, chuyển mục tiêu sang chuyến bay thay thế chẳng hạn như QH118 và tiếp tục hoàn tất chu trình đặt vé.

3) ĐÁNH GIÁ THỰC NGHIỆM & PHÂN TÍCH KẾT QUẢ



Hình 2: Bảng kết quả đánh giá thực nghiệm so sánh 3 mẫu thiết kế qua 3 kịch bản kiểm thử từ file benchmark.py

❖ **Bảng kết quả thực nghiệm thu được từ hệ thống:**

Kịch bản kiểm thử	Mẫu thiết kế	Kết quả thực thi	Số vòng lặp (Turns)	Đánh giá hành vi hệ thống
Kịch bản 1: Môi trường thuận lợi (Happy Path)	ReAct	SUCCESS	5	Chạy qua đầy đủ 5 bước suy luận - hành động.
	Plan-then-Execute	SUCCESS	4	Thực thi kế hoạch tuần tự, chi phí bước tối ưu.
	Hybrid	SUCCESS	4	Thực hiện trơn tru theo kế hoạch ban đầu.
Kịch bản 2: VN122 hết chỗ (Môi trường đổi)	ReAct	ABORT_LOOP	3	Lặp lại hành động tìm kiếm, bị LoopDetector ngắt an toàn.

	Plan-then-Execute	PLAN_BROKEN	2	Gãy kế hoạch ngay tại bước kiểm tra do hết ghế.
	Hybrid	SUCCESS	5	Kích hoạt Re-planning, chuyển sang QH118 và thành công
Kịch bản 3: Kiểm quyền Harness (Chặn hành động)	ReAct	WAIT_APPROVAL	3	PermissionGuard chặn vé không hoàn tiền, xuất Handoff.
	Plan-then-Execute	SUCCESS	4	Bỏ qua kiểm quyền động.
	Hybrid	SUCCESS	4	Chạy luồng chuẩn.

4) Nhận xét và phân tích chuyên sâu:

❖ Khả năng thích ứng với môi trường biến động:

- **Plan-then-Execute:** Bộc lộ sự cứng nhắc khi môi trường không giống như giả định. Chỉ cần 1 lỗi nhỏ ở khâu kiểm tra ghế là hệ thống sẽ dừng hoạt động.
- **Hybrid:** Thể hiện tính ổn định cao nhất khi mà duy trì được tính tổ chức của bản kế hoạch nhưng vẫn có khả năng tự phục hồi nhờ nhánh Re-planning.

❖ Vai trò sống còn của Harness Code:

- Nếu không có LoopDetector, mẫu ReAct trong Kịch bản 2 sẽ tiếp tục lặp lại các lệnh gọi tool cho đến khi cạn kiệt ngân sách tối đa và gây lãng phí chi phí token.
- Nếu không có PermissionGuard, Agent sẽ tự ý thực hiện đặt các loại vé không được hoàn tiền hoặc tự ý thanh toán thẻ mà không có sự kiểm duyệt của con người.

5) KẾT LUẬN

- Việc xây dựng một AI Agent thực tế không chỉ dừng lại ở việc kết nối Prompt với LLM, mà nằm ở việc thiết kế Harness Code vững chắc.
- Harness là chốt chặn đảm bảo an toàn cho các tác vụ nhạy cảm, kiểm định điều kiện dừng độc lập khách quan và quản lý ngân sách vận hành của hệ thống.